Table des matières

Exercices corrigés 05 — CTE et récursivité 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 5
Mises en situation 5
Corrigé 7

Exercices corrigés 05 — CTE et récursivité

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez cinq heures pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks` (schémas `sales`, `production`). Un énoncé marqué `[OLAP]` se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La définition. En une phrase, qu'est-ce qu'une CTE, et combien de temps existe-t-elle ?
2. Le mot-clé. Où s'écrit le `WITH` dans une requête, et combien peut-on en écrire pour une même requête, quel que soit le nombre de CTE ?
3. Le chaînage. Comment enchaîne-t-on trois CTE, et une CTE peut-elle lire celle qui la suit ?
4. L'astuce de débogage. Vous avez un pipeline de quatre CTE et le résultat final est faux. Quelle manipulation en une ligne vous permet d'inspecter n'importe quelle étape ?
5. Les trois outils. Remplissez de tête : sous-requête, CTE et vue, quelle est la portée de chacune, laquelle persiste, et quand choisir chacune ?

© 2026 iAlexMG. Tous droits réservés.

6. Réutiliser. Peut-on citer deux fois la même CTE dans le `SELECT` final ? Et une sous-requête, peut-on la citer deux fois sans la réécrire ?
7. Les trois morceaux. Nommez les trois parties d'une CTE récursive, dans l'ordre où on les écrit.
8. `UNION ALL` ou `UNION`. Pourquoi écrit-on `UNION ALL` entre l'ancre et la partie récursive, et que changerait un `UNION` simple ?
9. La condition d'arrêt. Qu'est-ce qui fait qu'une CTE récursive s'arrête ? Donnez les deux formes que prend cette condition.
10. Le mot-clé `RECURSIVE`. Sur quel moteur est-il obligatoire, sur lequel est-il facultatif, et que faut-il écrire pour être portable ?
11. Le message qui trompe. Sur PostgreSQL, vous oubliez `RECURSIVE`. Quel genre d'erreur obtenez-vous, et pourquoi ce message n'oriente-t-il pas vers la vraie cause ?
12. La série générée. À quoi sert une série de valeurs fabriquée sans table ? Donnez l'usage principal en analytique.
13. Combler les trous. Décrivez en trois étapes le mécanisme qui fait réapparaître les mois sans vente dans un rapport mensuel.
14. `[OLTP]` Le raccourci. Quelle fonction de PostgreSQL remplace la CTE récursive pour générer une série d'entiers, et pourquoi apprend-on quand même la CTE ?
15. `[OLAP]` Le raccourci d'en face. Qu'en est-il sur l'entrepôt, et quelle seconde forme DuckDB offre-t-il ? Attention à la borne.
16. La boucle sans fin. Que se passe-t-il si la condition d'arrêt manque, sur chacun des deux moteurs de la formation ? Quelle parade se code, et pourquoi ne peut-on pas s'en remettre au moteur ?

1. La matérialisation. Qu'est-ce qui a changé pour les CTE à PostgreSQL 12, et quel mot-clé permet de revenir au comportement d'avant ?
2. Le mythe. « Une CTE, c'est plus rapide qu'une sous-requête. » Vrai, faux, ça dépend ? Justifiez en une phrase.
3. `[OLAP]` L'ancre d'un organigramme. Quelle condition attrape la racine dans `DimEmployee`, et pourquoi cette colonne est-elle nulle sur une seule ligne ?
4. Le niveau. Comment obtient-on la profondeur hiérarchique de chaque ligne, sans jamais compter les tours de récursion à la main ?
5. Le chemin. Comment construit-on une colonne qui affiche la chaîne complète depuis la racine ? Quel piège de type guette sur cette colonne ?
6. `[OLTP]` La nomenclature. Dans `production.billofmaterials`, quelles sont les deux colonnes qui font de cette table un arbre, et vers quelle table pointent-elles toutes les deux ?
7. `[OLTP]` La colonne de validité. Pourquoi filtre-t-on sur `enddate IS NULL` dans les deux morceaux de la récursion, et pas seulement dans l'ancre ?
8. Les deux terrains. L'entrepôt vous donne un organigramme à plat, la base transactionnelle un arbre avec des dates de validité. Qu'est-ce que cet écart vous apprend sur le travail d'alimentation d'un entrepôt ?

Questions pratiques

© 2026 iAlexMG. Tous droits réservés.

Écrivez la requête complète avant de regarder. Quand un énoncé existe en deux versions, faites les deux et comparez-les : c'est là que le contraste entre les deux mondes devient concret.

P1. `[OLAP]` La CTE de lisibilité. Reprenez le « top 3 des produits par sous-catégorie » du Cours 04, mais sans `QUALIFY` : calculez le rang dans une CTE, puis filtrez à l'extérieur. C'est la version portable du `QUALIFY`.

P2. Les carrés. Avec une CTE récursive, générez les nombres de 1 à 10 et leur carré. Cette requête doit tourner à l'identique des deux côtés.

P3. `[OLAP]` L'effectif par niveau. À partir de l'organigramme, affichez combien d'employés se trouvent à chaque niveau hiérarchique.

P4. `[OLAP]` Le chemin complet. Reprenez l'organigramme et ajoutez une colonne `chemin` qui affiche la chaîne de commandement, façon `Ken Sanchez > Brian Welcker > …`.

P5. `[OLTP]` L'effectif par niveau, l'autre voie. Sur la nomenclature du produit 999, affichez combien de composants se trouvent à chaque niveau. Comparez la forme de l'arbre avec celle de l'organigramme.

P6. `[OLTP]` Le chemin complet, l'autre voie. Sur la même nomenclature, ajoutez une colonne `chemin` qui affiche la suite des noms de composants depuis le vélo. Attention au type de la colonne construite par concaténation.

P7. Le pipeline à trois étages. Écrivez, sur votre terrain préféré, une requête à trois CTE chaînées : les ventes par mois, puis la moyenne de ces mois, puis les seuls mois qui dépassent cette moyenne d'au moins 20 pour cent. Une CTE par idée, pas une de plus.

P8. `[OLAP]` L'année sans trous. Reprenez le rapport mensuel de 2010 avec les douze mois, et ajoutez une colonne qui indique par « oui » ou « non » si le mois a eu des ventes. Le `COALESCE` ne suffit pas ici, dites pourquoi.

P9. `[OLTP]` Le calendrier de la première année. Faites la même chose sur la première année de votre portage, en repérant l'année par une requête plutôt qu'en l'écrivant en dur.

P10. `(avancé)` La récursion bornée. Reprenez une des deux hiérarchies et ajoutez une double sécurité : une borne de profondeur explicite, et un compteur qui vous dit à quel tour la récursion s'est arrêtée. Dites ensuite ce que vous feriez de cette information dans du code de production.

P11. `(avancé)` Les feuilles de l'arbre. Sur la nomenclature du produit 999, ne gardez que les composants qui n'ont eux-mêmes aucun composant, autrement dit les pièces élémentaires. Deux écritures sont possibles, l'une qui interroge de nouveau la table de nomenclature, l'autre qui reste à l'intérieur du résultat de la CTE. Donnez les deux et dites laquelle vous préférez.

P12. `(avancé)` Le cumul des quantités. Toujours sur la nomenclature du produit 999, calculez pour chaque composant la quantité réellement nécessaire à la fabrication d'un vélo, c'est-à-dire le produit des quantités le long du chemin depuis la racine. Dites pourquoi une simple somme des `perassemblyqty` donnerait un résultat faux.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

© 2026 iAlexMG. Tous droits réservés.

E1. Un analyste veut les mois au-dessus de la moyenne. Il écrit, sur l'entrepôt :

```
WITH mensuel AS (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
)
SELECT annee, mois, ventes
FROM mensuel
WHERE ventes > AVG(ventes);
```

Le moteur refuse la requête. Expliquez la vraie raison du refus, dites pourquoi l'auteur a cru que ça passerait, et donnez deux corrections : une avec une seconde CTE, une avec une fonction de fenêtrage du Cours 04.

E2. Une développeuse déroule la nomenclature sur PostgreSQL :

```
WITH nomenclature AS (
    SELECT componentid, 1 AS niveau
    FROM production.billofmaterials
    WHERE productassemblyid = 999
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
)
SELECT * FROM nomenclature;
```

Elle obtient `ERROR: relation "nomenclature" does not exist`. Elle en conclut que sa CTE est mal nommée et la renomme trois fois. Dites ce qui cloche vraiment, pourquoi le message l'égare, et donnez le correctif. Signalez au passage un second défaut de cette requête, indépendant du premier.

E3. Un rapport doit montrer les douze mois de l'année, y compris ceux sans vente :

```
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
)
SELECT mois.m, SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey
JOIN mois ON mois.m = d.MonthNumberOfYear
```


© 2026 iAlexMG. Tous droits réservés.

```sql
WHERE d.CalendarYear = 2010
GROUP BY mois.m
ORDER BY mois.m;
```

Le résultat ne contient toujours qu'une ligne. L'auteur ne comprend pas, puisqu'il a bien généré les douze mois. Expliquez, puis corrigez.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. La lisibilité est un critère technique. On dit souvent qu'une CTE ne sert qu'à « rendre le code joli », et donc que c'est du confort. Défendez la position inverse : expliquez en quoi la lisibilité d'une requête a des conséquences mesurables sur la qualité du travail, en vous appuyant sur des situations concrètes comme la relecture par un collègue, le débogage à six mois, ou la reprise d'une requête par quelqu'un d'autre. Terminez en disant où se situe la limite, c'est-à-dire à partir de quand découper en CTE nuit plutôt qu'il n'aide.

B. Récursion et confiance. Une CTE récursive est le seul outil de ce cours qui peut faire tomber un serveur. Expliquez le mécanisme du danger, puis décrivez la discipline que vous adopteriez avant de laisser une requête récursive tourner sur une base de production. Distinguez ce qui relève du code, ce qui relève des garde-fous du moteur, et ce qui relève de la connaissance des données.

C. L'arbre et l'entrepôt. Vous avez déroulé un organigramme sur l'entrepôt et une nomenclature sur le transactionnel. Expliquez, à quelqu'un qui bâtit un entrepôt, quelles décisions se prennent au moment d'y faire entrer une hiérarchie : que garde-t-on, que met-on à plat, que perd-on. Prenez position sur le cas des dates de validité, et dites ce que vous en feriez.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Menuiserie Sainte-Adèle, le coût réel d'une commande. L'atelier fabrique des meubles sur mesure. Chaque meuble a une nomenclature à plusieurs niveaux, et chaque pièce élémentaire porte un coût unitaire dans une table de référence. On vous demande le coût de revient matière d'un meuble donné, c'est-à-dire la somme des coûts de toutes ses pièces élémentaires, chacune multipliée par la quantité réellement nécessaire compte tenu de sa place dans l'arbre.

Décrivez la requête : où va la récursion, comment se propage la quantité, et à quel moment se fait la multiplication par le coût. Le point difficile est la quantité cumulée le long du chemin : expliquez pourquoi une somme ne convient pas, et ce qui convient. Terminez en disant ce que vous faites si un composant apparaît à deux endroits de l'arbre du même meuble.

M2. Réseau de cliniques Rive-Sud, l'organigramme qui bouge. Le réseau compte 900 employés répartis sur onze établissements, avec une hiérarchie qui change souvent, et une table du personnel qui porte une colonne `superviseur_id` et deux colonnes de validité, `debut` et `fin`. La direction veut trois choses : l'organigramme au 1er janvier de l'an dernier, l'organigramme d'aujourd'hui, et la liste des personnes dont le niveau hiérarchique a changé entre les deux dates.

© 2026 iAlexMG. Tous droits réservés.

Écrivez le raisonnement complet. Précisez comment une date se glisse dans une CTE récursive, et pourquoi il vaut mieux la passer une fois plutôt que de la répéter dans les deux morceaux. Dites ensuite ce que vous faites des personnes arrivées ou parties entre les deux dates, qui n'existent que d'un côté. Terminez sur le point qui décide de tout : cette requête doit-elle vivre dans l'application ou dans un entrepôt, et qu'est-ce qui tranche ?

M3. `(avancé)` Coopérative de transport Chaudière, la tournée qui se referme. La coopérative modélise son réseau comme une table d'arêtes, `depart` et `arrivee`, entre points de collecte. On vous demande tous les trajets possibles entre deux points donnés, avec le nombre d'étapes. Le réseau contient des boucles : certains points se rejoignent dans les deux sens.

Écrivez la stratégie complète. Le cœur du problème est que la récursion, sur un graphe cyclique, ne s'arrête pas d'elle-même : décrivez précisément les deux mécanismes de protection, celui qui borne la profondeur et celui qui empêche de repasser par un point déjà visité, et dites pourquoi le second est le seul qui soit vraiment correct. Comparez ensuite le coût de cette requête sur un entrepôt et sur une base transactionnelle, puis répondez à la question qui fâche : est-ce que le SQL est le bon outil pour ce problème, et si non, à partir de quelle taille de réseau changez-vous d'avis ?

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Une CTE est un résultat intermédiaire nommé en tête de requête avec `WITH`. Elle n'existe que le temps de l'exécution de la requête qui la déclare : rien n'est stocké, rien à nettoyer.

Théorie : `Theorie_05_SQL.md`, section 2.

2. Le `WITH` s'écrit tout en tête, avant le `SELECT` final. Il n'y en a qu'un seul par requête, quel que soit le nombre de CTE : les suivantes se séparent par une virgule.

Théorie : `Theorie_05_SQL.md`, section 2.

3. On les sépare par des virgules et chacune peut lire celles qui la précèdent. Une CTE ne peut pas lire celle qui la suit : les définitions se résolvent dans l'ordre d'écriture. La seule exception est une CTE récursive, qui se lit elle-même, ce qui est un cas différent.

Théorie : `Theorie_05_SQL.md`, section 3.

4. On remplace le `SELECT` final par `SELECT * FROM une_etape;`. C'est l'avantage pratique décisif du pipeline de CTE sur les sous-requêtes empilées, où il faudrait extraire la parenthèse à la main.

Théorie : `Theorie_05_SQL.md`, section 3.

5. Trois portées, trois durées de vie.

Outil	Portée	Persiste	Quand la choisir
Sous-requête	l'endroit exact où elle est écrite	non	un calcul court, utilisé une seule fois
CTE	toute la requête qui suit	non	lisibilité, réutilisation dans la même requête, récursivité
Vue	toutes les requêtes futures	oui, c'est un objet de la base	une logique partagée entre plusieurs requêtes

Théorie : `Theorie_05_SQL.md`, section 4.

6. Oui pour la CTE : c'est même une de ses raisons d'être, et le moteur ne recalcule pas nécessairement deux fois. Non pour la sous-requête : il faudrait la réécrire intégralement au second endroit, avec le risque que les deux copies divergent à la première modification.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_SQL.md`, section 4.

7. L'ancre, puis `UNION ALL`, puis la partie récursive, celle qui se réfère au nom de la CTE.

Théorie : `Theorie_05_SQL.md`, section 5.

8. `UNION ALL` empile sans dédoublonner, ce qui est ce qu'on veut : chaque tour ajoute une couche. Un `UNION` simple imposerait au moteur de comparer chaque nouvelle ligne à toutes les précédentes, ce qui coûte cher, et il supprimerait des lignes légitimement identiques, par exemple deux composants de même nom à des endroits différents de l'arbre.

Théorie : `Theorie_05_SQL.md`, section 5.

9. La récursion s'arrête quand la partie récursive ne renvoie plus aucune ligne. Cela arrive de deux façons : une condition explicite qui devient fausse (`WHERE n < 12`), ou l'épuisement naturel des données, quand la jointure ne trouve plus d'enfant à rattacher, ce qui est le cas d'un arbre fini.

Théorie : `Theorie_05_SQL.md`, section 5.

10. Obligatoire sur les deux moteurs de la formation. DuckDB le dit d'ailleurs sans détour quand on l'oublie : « Circular reference to CTE, use WITH RECURSIVE to use recursive CTEs ». Quelques plateformes s'en passent, dont Snowflake d'après sa documentation, mais l'écrire ne coûte rien et règle la question partout.

Théorie : `Theorie_05_SQL.md`, section 5.

11. Le moteur signale que la relation n'existe pas, du genre `ERROR: relation "nombres" does not exist`. Le message égare parce qu'il parle de nom, ce qui suggère une faute de frappe ou un schéma manquant. La vraie cause est que, sans `RECURSIVE`, PostgreSQL n'autorise pas une CTE à se citer elle-même : le nom n'est tout simplement pas encore visible à l'intérieur de sa propre définition.

Théorie : `Theorie_05_SQL.md`, section 5.

12. À fabriquer un axe complet là où les données n'en fournissent qu'un partiel. L'usage principal en analytique est de combler les trous d'un rapport périodique, pour que les périodes sans événement apparaissent quand même, à zéro.

Théorie : `Theorie_05_SQL.md`, section 6.

13. Trois étapes. On génère l'axe complet, les douze mois par exemple. On agrège les données réelles séparément, dans leur propre CTE. On raccroche l'un à l'autre par un `LEFT JOIN` depuis l'axe vers les données, avec un `COALESCE` pour remplacer par zéro les périodes sans ligne. Le sens du `LEFT JOIN` est le point critique : c'est l'axe qui commande.

Théorie : `Theorie_05_SQL.md`, section 6.

14. `[OLTP]` `generate_series(1, 12)`. On apprend quand même la CTE récursive parce qu'elle est portable, parce qu'elle est ce qu'on vous demandera d'écrire en entretien, et parce qu'elle reste disponible dans un contexte où l'on ne veut pas dépendre d'une fonction propre au moteur.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_SQL.md`, section 6.

15. `[OLAP]` Le même, `generate_series(1, 12)` : c'est l'une des surprises agréables du double chemin, le raccourci ne change pas de nom d'un terrain à l'autre. DuckDB en offre une seconde forme, `range(1, 13)`, dont la borne haute est exclue, d'où le 13 pour obtenir douze valeurs. Cette borne exclue est la seule chose à retenir ici, parce qu'elle donne onze lignes au lieu de douze à qui recopie sans lire. D'autres plateformes écrivent tout autrement : sur Snowflake, d'après sa documentation, `TABLE(GENERATOR(ROWCOUNT => 12))` ne rend que douze lignes vides, et c'est `ROW_NUMBER()` qui en fait les nombres de 1 à 12.

Théorie : `Theorie_05_SQL.md`, section 6.

16. Sans condition d'arrêt, la partie récursive renvoie toujours des lignes, et aucun de nos deux moteurs n'y met le holà. PostgreSQL n'impose aucune limite de profondeur : la requête tourne jusqu'à épuiser la mémoire du serveur, ce qui est un incident et non une erreur de requête. DuckDB fait de même, ce qui a été vérifié : une récursion sans borne y tournait toujours après trois minutes, sans message et sans résultat, et il a fallu l'interrompre à la main.

La parade se code donc, et elle se code toujours : une condition qui finira par être fausse, plus une borne de sécurité du genre `AND niveau < 10` quand l'arbre vient de données qu'on ne maîtrise pas. On ne peut pas s'en remettre au moteur, d'abord parce que le nôtre ne coupe pas, ensuite parce qu'une limite de profondeur n'est de toute façon pas une protection : c'est un pis-aller qui transforme une boucle en erreur, à un endroit arbitraire, sans dire laquelle des lignes était fautive. Certaines plateformes en posent une, dont Snowflake d'après sa documentation, réglable par `MAX_RECURSIONS`.

Théorie : `Theorie_05_SQL.md`, section 8.

17. Avant la version 12, PostgreSQL matérialisait toute CTE de force : elle était calculée et stockée à part, ce qui bloquait des optimisations, notamment la remontée d'un filtre à l'intérieur de la CTE. Depuis la 12, la règle dépend du nombre de lectures, et c'est la nuance que beaucoup d'explications sautent : une CTE citée une seule fois est intégrée, une CTE citée deux fois ou plus reste matérialisée par défaut, le moteur jugeant qu'il vaut mieux la calculer une fois que deux. Le mot-clé `AS MATERIALIZED` force l'ancien comportement dans tous les cas, et `AS NOT MATERIALIZED` force l'inverse.

La vérification tient en deux `EXPLAIN` : une CTE lue deux fois apparaît dans le plan comme un nœud `CTE Scan`, et la même déclarée `NOT MATERIALIZED` en disparaît, fondue dans la requête qui la lit.

Théorie : `Theorie_05_SQL.md`, section 8.

18. Ça dépend, et le plus souvent c'est faux comme raisonnement. Une CTE est d'abord une aide à la lecture. Mais dire qu'elle n'est jamais un levier de performance serait faux aussi, et la question mérite mieux qu'un slogan.

Sur une CTE lue une seule fois, l'affirmation tient : les deux moteurs l'intègrent et le plan est le même qu'avec une sous-requête. Sur une CTE lue deux fois ou plus, PostgreSQL la matérialise par défaut, et il a souvent raison : calculer une fois ce qu'on lit deux fois est un gain réel. C'est d'ailleurs la raison pour laquelle on écrit des CTE, et elle est donc bien, dans ce cas précis, un levier de performance.

Le mauvais réflexe n'est pas d'espérer un gain, c'est de ne pas savoir lequel des deux comportements on obtient. Les deux mots-clés, `AS MATERIALIZED` et `AS NOT MATERIALIZED`, servent exactement à cela : décider au lieu de subir. La matérialisation aide quand un calcul coûteux sert plusieurs fois, et nuit quand elle empêche un filtre de descendre à l'intérieur.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_SQL.md`, section 8.

19. `[OLAP]` `WHERE ParentEmployeeKey IS NULL`. Cette colonne est nulle sur une seule ligne parce qu'une hiérarchie d'entreprise a une seule racine : la personne qui n'a pas de supérieur. Si la requête ramenait plusieurs lignes au niveau 1, il faudrait s'inquiéter, soit d'une donnée douteuse, soit d'une organisation à plusieurs têtes qu'il faudrait alors traiter comme une forêt et non comme un arbre.

Théorie : `Theorie_05_SQL.md`, section 7.

20. On pose un `1` littéral dans l'ancre, et `o.niveau + 1` dans la partie récursive. La profondeur se construit donc toute seule, en même temps que les lignes, sans que personne n'ait à savoir combien de tours la récursion fera.

Théorie : `Theorie_05_SQL.md`, section 7.

21. Dans l'ancre, on initialise la colonne avec le nom de la racine. Dans la partie récursive, on concatène le chemin du parent, un séparateur, et le nom de l'enfant. Le piège de type est que c'est l'ancre qui fixe le type de la colonne pour toute la récursion, et nos deux moteurs n'en tirent pas les mêmes conséquences.

DuckDB élargit tout seul : une ancre qui pose un `VARCHAR(10)` laisse passer des chemins de vingt et un caractères sans rien dire. PostgreSQL, lui, refuse la requête avant de la lancer, et son message nomme exactement le problème : « dans la requête récursive, la colonne 2 a le type character varying(10) dans le terme non récursif mais le type global character varying », suivi de l'astuce « Convertissez la sortie du terme non récursif dans le bon type ».

Des deux comportements, celui de PostgreSQL est le plus confortable, parce qu'il échoue au moment où l'on peut encore corriger. La parade vaut donc des deux côtés et coûte une ligne : forcer un type large dès l'ancre, par un `CAST` explicite. On prend l'habitude sur le moteur permissif pour ne pas buter sur l'autre.

Théorie : `Theorie_05_SQL.md`, section 7. Théorie : `Theorie_02_SQL.md`, section 4.

22. `[OLTP]` `productassemblyid`, l'assemblage, et `componentid`, la pièce qui y entre. Les deux pointent vers `production.product`, ce qui est la marque d'un arbre : la table relie une entité à elle-même par l'intermédiaire d'une table de liens.

Théorie : `Theorie_05_SQL.md`, sections 7 et 9.

23. `[OLTP]` Parce que la récursion se propage. Filtrer seulement dans l'ancre écarterait les nomenclatures périmées au premier niveau, mais tous les niveaux suivants les feraient rentrer par la partie récursive. Une erreur de filtre dans une récursion ne reste jamais locale : elle contamine toute la descendance.

Théorie : `Theorie_05_SQL.md`, section 9.

24. Que la propreté d'un entrepôt n'est pas gratuite, elle est produite. Quelqu'un a écrit l'alimentation qui a choisi une version de référence, mis la hiérarchie à plat dans une colonne parent, et jeté les lignes périmées. La difficulté n'a pas disparu, elle a été payée une fois, en amont, au lieu d'être repayée par chaque analyste à chaque requête. C'est exactement ce que vend le métier d'ingénieur analytique.

Théorie : `Theorie_05_SQL.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. [OLAP]

```sql
WITH classe AS (
    SELECT ProductSubcategoryKey, EnglishProductName, ListPrice,
           ROW_NUMBER() OVER (PARTITION BY ProductSubcategoryKey
                              ORDER BY ListPrice DESC) AS rang
    FROM dbo.DimProduct
    WHERE ListPrice > 0 AND ProductSubcategoryKey IS NOT NULL
)
SELECT ProductSubcategoryKey, EnglishProductName, ListPrice
FROM classe
WHERE rang <= 3
ORDER BY ProductSubcategoryKey, ListPrice DESC;
```

C'est mot pour mot ce que `QUALIFY` écrit à votre place, et c'est la seule écriture disponible sur PostgreSQL.

Théorie : `Theorie_05_SQL.md`, section 2.

P2.

```sql
WITH RECURSIVE nombres AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nombres WHERE n < 10
)
SELECT n, n * n AS carre FROM nombres;
```

Théorie : `Theorie_05_SQL.md`, sections 5 et 6.

P3. [OLAP]

```sql
WITH RECURSIVE organigramme AS (
    SELECT EmployeeKey, ParentEmployeeKey, 1 AS niveau
    FROM dbo.DimEmployee
    WHERE ParentEmployeeKey IS NULL
    UNION ALL
    SELECT e.EmployeeKey, e.ParentEmployeeKey, o.niveau + 1
    FROM dbo.DimEmployee AS e
    JOIN organigramme    AS o ON e.ParentEmployeeKey = o.EmployeeKey
)
SELECT niveau, COUNT(*) AS nb_employes
FROM organigramme
```


© 2026 iAlexMG. Tous droits réservés.

```sql
GROUP BY niveau
ORDER BY niveau;
```

Théorie : `Theorie_05_SQL.md`, section 7.

P4. [OLAP]

```sql
WITH RECURSIVE organigramme AS (
    SELECT EmployeeKey, ParentEmployeeKey,
           CAST(FirstName || ' ' || LastName AS VARCHAR(1000)) AS chemin
    FROM dbo.DimEmployee
    WHERE ParentEmployeeKey IS NULL
    UNION ALL
    SELECT e.EmployeeKey, e.ParentEmployeeKey,
           CAST(o.chemin || ' > ' || e.FirstName || ' ' || e.LastName AS VARCHAR(1000))
    FROM dbo.DimEmployee AS e
    JOIN organigramme    AS o ON e.ParentEmployeeKey = o.EmployeeKey
)
SELECT chemin FROM organigramme ORDER BY chemin;
```

Le `CAST` explicite dans les deux morceaux est la précaution de la question 21. DuckDB s'en passe volontiers, la même requête sans `CAST` y rendant les mêmes 296 lignes et un chemin de 81 caractères au plus long ; PostgreSQL est plus strict dès que l'ancre pose un type étroit. Prendre l'habitude coûte une ligne, et elle se prend sur le moteur permissif.

Théorie : `Theorie_05_SQL.md`, section 7.

P5. [OLTP]

```sql
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT niveau, COUNT(*) AS nb_composants
FROM nomenclature
GROUP BY niveau
ORDER BY niveau;
```


© 2026 iAlexMG. Tous droits réservés.

Les deux arbres n'ont pas la même forme, et c'est instructif. L'organigramme est large et peu profond, parce qu'un manager encadre beaucoup de monde. La nomenclature est étroite et plus profonde, parce qu'un assemblage se décompose en quelques sous-ensembles, eux-mêmes décomposés.

Théorie : `Theorie_05_SQL.md`, sections 7 et 9.

P6. `[OLTP]`

```sql
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau,
           CAST(p.name AS VARCHAR(1000)) AS chemin
    FROM production.billofmaterials AS b
    JOIN production.product AS p ON p.productid = b.componentid
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1,
           CAST(n.chemin || ' > ' || p.name AS VARCHAR(1000))
    FROM production.billofmaterials AS b
    JOIN nomenclature       AS n ON b.productassemblyid = n.componentid
    JOIN production.product AS p ON p.productid = b.componentid
    WHERE b.enddate IS NULL
)
SELECT niveau, chemin FROM nomenclature ORDER BY chemin;
```

La jointure vers `production.product` apparaît deux fois, une par morceau. C'est normal et c'est le prix du nom lisible : la table de liens ne porte que des identifiants.

Théorie : `Theorie_05_SQL.md`, sections 7 et 9.

P7. Version entrepôt, la version transactionnelle ne change que la première CTE.

```sql
WITH mensuel AS (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
),
moyenne AS (
    SELECT AVG(ventes) AS ventes_moyens FROM mensuel
),
au_dessus AS (
    SELECT m.annee, m.mois, m.ventes, mo.ventes_moyens
    FROM mensuel AS m
```


© 2026 iAlexMG. Tous droits réservés.

```
    CROSS JOIN moyenne AS mo
    WHERE m.ventes > mo.ventes_moyens * 1.20
)
SELECT annee, mois, ventes,
       ventes / ventes_moyens - 1 AS ecart_relatif
FROM au_dessus
ORDER BY ecart_relatif DESC;
```

Théorie : `Theorie_05_SQL.md`, section 3.

P8. [OLAP]

```
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
),
ventes_2010 AS (
    SELECT d.MonthNumberOfYear AS mois, SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    WHERE d.CalendarYear = 2010
    GROUP BY d.MonthNumberOfYear
)
SELECT mois.m AS mois,
       COALESCE(v.ventes, 0) AS ventes,
       CASE WHEN v.mois IS NULL THEN 'non' ELSE 'oui' END AS a_vendu
FROM mois
LEFT JOIN ventes_2010 AS v ON v.mois = mois.m
ORDER BY mois.m;
```

Le `COALESCE` ne suffit pas parce qu'il détruit précisément l'information qu'on cherche. Une fois le zéro posé, un mois sans aucune vente et un mois dont les ventes s'annulent, remboursements compris, sont devenus indiscernables. Le test doit donc porter sur l'absence de ligne, donc sur `v.mois IS NULL`, et il doit être écrit avant que le `COALESCE` ne fasse son travail. C'est un réflexe général : on teste la présence sur la clé de jointure, jamais sur la mesure.

Théorie : `Theorie_05_SQL.md`, section 6. Théorie : `Theorie_02_SQL.md`, section 3.

P9. [OLTP]

```
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
```


© 2026 iAlexMG. Tous droits réservés.

```
),
premiere_annee AS (
    SELECT EXTRACT(YEAR FROM MIN(orderdate))::int AS annee
    FROM sales.salesorderheader
),
ventes_annee AS (
    SELECT EXTRACT(MONTH FROM h.orderdate)::int AS mois, SUM(d.linetotal) AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    CROSS JOIN premiere_annee AS pa
    WHERE EXTRACT(YEAR FROM h.orderdate) = pa.annee
    GROUP BY 1
)
SELECT mois.m AS mois,
       COALESCE(v.ventes, 0) AS ventes,
       CASE WHEN v.mois IS NULL THEN 'non' ELSE 'oui' END AS a_vendu
FROM mois
LEFT JOIN ventes_annee AS v ON v.mois = mois.m
ORDER BY mois.m;
```

Repérer l'année par une requête plutôt qu'en dur n'est pas un raffinement d'esthète. Une valeur écrite en dur est une bombe à retardement : elle est juste le jour où on l'écrit, et elle devient fausse le jour où quelqu'un recharge la base avec un autre jeu de dates, sans que rien ne le signale.

Théorie : `Theorie_05_SQL.md`, sections 6 et 9.

P10. (avancé) Sur la nomenclature, avec les deux sécurités demandées.

```
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
      AND n.niveau < 10                      -- borne de profondeur explicite
)
SELECT MAX(niveau) AS profondeur_atteinte, COUNT(*) AS lignes
FROM nomenclature;
```

En production, on ne se contente pas de regarder cette profondeur, on la surveille. Si elle atteint la borne, c'est le signal que le résultat est tronqué, donc faux, et non pas simplement limité. La bonne pratique est de placer la borne nettement au-dessus de la profondeur attendue, quatre ici, puis de faire échouer

© 2026 iAlexMG. Tous droits réservés.

bruyamment le traitement quand elle est atteinte, parce qu'une donnée qui boucle est un incident de données à corriger à la source, pas un cas à absorber en silence.

Théorie : `Theorie_05_SQL.md`, sections 5 et 8.

P11. (avancé) Deux écritures.

```sql
-- Version 1 : on interroge de nouveau la table de nomenclature
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT DISTINCT n.niveau, p.name AS piece
FROM nomenclature AS n
JOIN production.product AS p ON p.productid = n.componentid
WHERE NOT EXISTS (
    SELECT 1 FROM production.billofmaterials AS b2
    WHERE b2.productassemblyid = n.componentid AND b2.enddate IS NULL
)
ORDER BY n.niveau, piece;
```

```sql
-- Version 2 : on reste dans le resultat de la CTE
WITH RECURSIVE nomenclature AS (
    SELECT b.productassemblyid, b.componentid, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.productassemblyid, b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT DISTINCT n.niveau, p.name AS piece
FROM nomenclature AS n
JOIN production.product AS p ON p.productid = n.componentid
WHERE n.componentid NOT IN (SELECT productassemblyid FROM nomenclature)
ORDER BY n.niveau, piece;
```


© 2026 iAlexMG. Tous droits réservés.

Je préfère la première. La seconde a l'air plus élégante, mais elle ment sur ce qu'elle calcule : elle déclare feuille tout composant qui n'apparaît pas comme parent à l'intérieur de cet arbre-ci, alors qu'il pourrait très bien avoir des composants ailleurs, hors de la branche explorée. La première pose la vraie question, celle qui porte sur la table entière. Elle a aussi un défaut à connaître : `NOT IN` se fait piéger par un `NULL`, et `productassemblyid` peut être nul dans cette table, ce qui suffirait à vider le résultat de la seconde version. C'est le piège du Cours 01, qui revient là où on ne l'attendait plus.

Théorie : `Theorie_05_SQL.md`, section 7.

P12. `(avancé)` La quantité se propage par multiplication, pas par addition.

```sql
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau,
           CAST(b.perassemblyqty AS NUMERIC) AS qte_cumulee
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1,
           CAST(n.qte_cumulee * b.perassemblyqty AS NUMERIC)
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT p.name AS composant, SUM(n.qte_cumulee) AS qte_pour_un_velo
FROM nomenclature AS n
JOIN production.product AS p ON p.productid = n.componentid
GROUP BY p.name
ORDER BY qte_pour_un_velo DESC;
```

Une somme des `perassemblyqty` serait fausse parce que ces quantités sont relatives à leur assemblage, pas au produit fini. Si un vélo porte deux roues et qu'une roue porte trente-six rayons, il faut soixante-douze rayons, pas trente-huit. La quantité se compose le long du chemin, donc elle se multiplie. La somme du `SELECT` final, elle, est légitime et sert à autre chose : elle totalise les apparitions d'une même pièce dans plusieurs branches de l'arbre.

Les deux `CAST` ne sont pas décoratifs, et c'est ici que la question 21 se paie pour de vrai. `perassemblyqty` est un `numeric(8,2)`, donc l'ancre pose ce type pour la colonne ; la multiplication de la partie récursive, elle, produit un `numeric` sans précision annoncée. PostgreSQL refuse alors la requête entière, sur « la colonne 3 a le type numeric(8,2) dans le terme non récursif mais le type global numeric », et il a raison de la refuser : sans cette vérification, la colonne se serait retrouvée bornée à deux décimales par une ancre qui ne décidait rien. Élargir le type dès l'ancre règle la question. Voilà la différence entre une précaution de style et une précaution nécessaire : celle-ci l'est.

Théorie : `Theorie_05_SQL.md`, section 7.

Diagnostics

© 2026 iAlexMG. Tous droits réservés.

E1. Le moteur refuse parce qu'une fonction d'agrégat n'a pas sa place dans un `WHERE`. La raison est celle du Cours 01, l'ordre d'exécution : le `WHERE` filtre ligne à ligne, avant toute agrégation, donc au moment où il s'évalue la moyenne n'existe pas encore et ne peut pas exister, puisqu'elle suppose d'avoir déjà parcouru toutes les lignes.

L'auteur a cru que ça passerait parce que `mensuel` ressemble à une table déjà calculée. C'est vrai du point de vue de la lecture, et faux du point de vue de l'exécution : une CTE n'est pas une table matérialisée, et surtout la moyenne dont il a besoin porte sur le résultat de la CTE, pas sur une ligne de la CTE. Le raisonnement à retenir est qu'il lui manque un niveau, pas un mot-clé.

```sql
-- Correction 1 : une seconde CTE
WITH mensuel AS ( /* ... inchange ... */ ),
moyenne AS (
    SELECT AVG(ventes) AS ventes_moyens FROM mensuel
)
SELECT m.annee, m.mois, m.ventes
FROM mensuel AS m
CROSS JOIN moyenne
WHERE m.ventes > moyenne.ventes_moyens;

-- Correction 2 : une fenetre du Cours 04, qui evite le second niveau
WITH mensuel AS ( /* ... inchange ... */ )
SELECT annee, mois, ventes
FROM mensuel
QUALIFY ventes > AVG(ventes) OVER ();
```

La seconde correction ne vaut que sur l'entrepôt, `QUALIFY` n'existant pas sur PostgreSQL, où il faudrait envelopper la fenêtre dans une sous-requête (Cours 04, section 7). Elle mérite d'être vue parce qu'elle montre que les deux outils du Cours 04 et du Cours 05 répondent souvent à la même question : la fenêtre apporte le contexte sans niveau supplémentaire, la CTE apporte un niveau lisible.

Théorie : `Theorie_05_SQL.md`, sections 1 et 2.

E2. Il manque `RECURSIVE`. Sans ce mot-clé, PostgreSQL ne rend pas le nom de la CTE visible à l'intérieur de sa propre définition, donc la partie récursive référence un nom qui n'existe pas encore. Le message égare parce qu'il parle de relation inexistante, ce qui oriente vers une faute de nom ou un schéma manquant, alors que le nom est parfaitement correct. Renommer la CTE ne pouvait donc rien donner, et c'est même le pire des symptômes : trois essais infructueux qui confirment chacun le mauvais diagnostic.

```sql
WITH RECURSIVE nomenclature AS (
    SELECT componentid, 1 AS niveau
    FROM production.billofmaterials
    WHERE productassemblyid = 999 AND enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
```


© 2026 iAlexMG. Tous droits réservés.

```
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL AND n.niveau < 10
)
SELECT * FROM nomenclature;
```

Le second défaut, indépendant du premier, est l'absence de filtre sur `enddate`. La requête d'origine aurait déroulé les nomenclatures périmées en même temps que les courantes, et le résultat aurait été plausible, donc dangereux, puisque rien n'aurait signalé l'erreur. On en profite pour ajouter la borne de profondeur, qui manquait aussi.

Un mot sur les messages, parce que la même faute se diagnostique très inégalement selon le terrain. La même CTE sans `RECURSIVE`, lancée sur l'entrepôt, donne : « Circular reference to CTE "nomenclature", use WITH RECURSIVE to use recursive CTEs ». Le message nomme la cause et donne la correction. Sur PostgreSQL, il faut la deviner. Quand vous avez le choix du terrain pour mettre une requête au point, le moteur qui parle le mieux vous fait gagner du temps, et ce n'est pas toujours celui sur lequel la requête finira.

Théorie : `Theorie_05_SQL.md`, section 5.

E3. Le `JOIN` vers `mois` est une jointure interne, donc il ne garde que les mois présents des deux côtés. Or les onze mois sans vente n'existent pas dans la table de faits : ils sont éliminés par la jointure avant même d'arriver au `GROUP BY`. Générer les douze mois ne sert à rien si on les fait ensuite passer par un filtre qui les supprime, et c'est exactement ce qu'est une jointure interne.

Le second problème est le sens de la construction. Il faut agréger d'abord, puis raccrocher l'axe complet à ce résultat agrégé par un `LEFT JOIN` partant de l'axe. Joindre l'axe directement à la table de faits ne peut pas marcher, même avec un `LEFT JOIN`, parce que le `WHERE` sur l'année porterait alors sur la table de droite, la table de faits, et rejetterait les `NULL`, ce qui redégraderait la jointure en interne. C'est le piège du `LEFT JOIN` du Cours 01, dans un habit nouveau.

```
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
),
ventes_2010 AS (
    SELECT d.MonthNumberOfYear AS mois, SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    WHERE d.CalendarYear = 2010
    GROUP BY d.MonthNumberOfYear
)
SELECT mois.m AS mois, COALESCE(v.ventes, 0) AS ventes
FROM mois
LEFT JOIN ventes_2010 AS v ON v.mois = mois.m
ORDER BY mois.m;
```

Théorie : `Theorie_05_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

Questions à développer

A. La lisibilité n'est pas du confort, c'est le premier facteur de fiabilité d'une requête, pour une raison simple : personne ne peut vérifier ce qu'il ne peut pas lire. Une sous-requête à trois niveaux se relit en reconstituant mentalement l'ordre d'évaluation, ce qui est un travail, et ce travail est refait par chaque personne qui touche au code. Une erreur logique, un `GROUP BY` oublié, un filtre au mauvais étage, se voit dans un pipeline de CTE nommées et se cache dans une imbrication.

Les situations concrètes le montrent mieux qu'un principe. En relecture, un collègue peut approuver une CTE nommée `ventes_mensuelles` parce qu'il en vérifie le contenu contre son nom ; il ne peut que faire semblant d'approuver une parenthèse de quinze lignes. Au débogage, on isole une étape en une ligne, alors qu'avec des sous-requêtes il faut découper à la main, avec le risque de déboguer une requête qui n'est plus tout à fait celle qui échoue. À six mois, le nom des étapes est souvent la seule documentation qui reste, et c'est la seule qui ne mente jamais, puisqu'elle s'exécute.

La limite existe. Découper en CTE une requête de dix lignes ajoute de la cérémonie sans rien apporter. Multiplier les CTE d'une ligne, qui ne font qu'aliaser une table, force le lecteur à faire des allers-retours pour reconstituer ce qu'une lecture linéaire aurait donné. La bonne mesure est qu'une CTE doit correspondre à une étape que vous sauriez nommer en français devant un collègue : si le nom que vous trouvez est `etape2`, c'est que le découpage est arbitraire.

Théorie : `Theorie_05_SQL.md`, sections 1 et 4.

B. Le danger tient à une propriété unique de la récursion : c'est la seule construction de ce cours dont le coût ne se déduit pas de la taille des données en entrée. Une jointure sur dix millions de lignes est lourde mais bornée ; une CTE récursive sur une table de mille lignes peut ne jamais s'arrêter, s'il existe un cycle. Et un cycle n'est pas une hypothèse d'école : dans un organigramme, il suffit que deux personnes se soient déclarées supérieures l'une de l'autre lors d'une réorganisation mal saisie.

La discipline se répartit sur trois plans. Dans le code, on borne toujours la profondeur par une condition explicite, même quand on croit l'arbre fini, et on traîne au besoin le chemin parcouru pour refuser d'y repasser. Du côté du moteur, on sait sur lequel on tourne, et sur les deux moteurs de cette formation la réponse est la même : ni DuckDB ni PostgreSQL ne coupent une boucle, ce qui veut dire qu'on n'essaie jamais une récursion nouvelle sans filet. Du côté des données, on regarde avant de dérouler : combien de racines, quelle profondeur attendue, y a-t-il une contrainte qui interdit les cycles ou seulement une convention. Une contrainte de base de données qui empêche un cycle vaut mieux que n'importe quelle prudence dans la requête, parce qu'elle protège aussi celui qui écrira la requête suivante.

Il reste une règle de conduite, qui n'est pas technique : on n'essaie pas une récursion nouvelle directement sur la base de production. On la met au point sur une copie, ou bornée à une seule racine, et on ne la libère qu'après avoir vu sa profondeur réelle.

Théorie : `Theorie_05_SQL.md`, sections 5 et 8.

C. Faire entrer une hiérarchie dans un entrepôt demande de répondre à trois questions, et chacune se paie plus tard si on la tranche mal.

Ce qu'on garde. Une hiérarchie a une forme, et cette forme peut se conserver telle quelle, par une colonne parent, ou se dérouler à l'avance en colonnes de niveau, `niveau_1`, `niveau_2`, et ainsi de suite. La colonne parent garde la souplesse et impose une récursion à chaque lecture. Le déroulement à plat rend la lecture triviale mais fige la profondeur, ce qui casse le jour où l'organisation ajoute un étage. Le choix se fait sur la stabilité de la hiérarchie, pas sur l'élégance.

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on met à plat. Les libellés lisibles, presque toujours. Un entrepôt qui oblige à joindre trois fois pour afficher le nom d'un supérieur trahit sa vocation. C'est aussi ce qu'a fait AdventureWorks DW en posant un simple `ParentEmployeeKey` là où le système source range un chemin de type `hierarchyid` : l'entrepôt a traduit une structure technique en une structure interrogeable.

Ce qu'on perd. Rien ici, et c'est la surprise : `DimEmployee` a gardé son historique, dans la dimension même, avec `StartDate`, `EndDate` et `Status`. Ce qu'on perd, c'est la simplicité qu'on croyait avoir gagnée, puisqu'il faut désormais choisir une date de lecture avant de parcourir l'arbre. Une hiérarchie recopiée telle qu'elle est aujourd'hui, elle, écraserait ce qu'elle était hier, et rendrait impossible toute question du genre « qui supervisait ce dossier à l'époque ». C'est là que je prends position sur les dates de validité : je les garde, en modélisant la hiérarchie en dimension à évolution lente, avec un début et une fin de validité sur chaque lien. Le coût est réel, chaque requête doit préciser à quelle date elle se place, et c'est précisément pour cette raison qu'on fournit à côté une vue de l'état courant, celle que quatre-vingt-dix pour cent des usages consommeront. Jeter la validité par simplicité est une décision qu'on ne peut pas défaire : les états passés, une fois écrasés, ne se reconstituent pas.

Théorie : `Theorie_05_SQL.md`, sections 7 et 9.

Mises en situation

M1. Menuiserie Sainte-Adèle.

La requête se construit en trois temps. Une CTE récursive descend la nomenclature du meuble demandé, en partant de ses composants directs et en propageant deux choses à chaque niveau : la profondeur, et surtout la quantité cumulée, obtenue en multipliant la quantité du parent par celle du lien courant. Un second niveau ne garde que les feuilles, c'est-à-dire les composants qui n'apparaissent nulle part comme assemblage. Un troisième joint ces feuilles à la table des coûts et multiplie la quantité cumulée par le coût unitaire, puis totalise.

La multiplication par le coût se fait donc en dernier, et seulement sur les feuilles. C'est un point qu'on rate souvent : appliquer le coût à tous les niveaux compterait plusieurs fois la même matière, une fois dans le sous-ensemble et une fois dans les pièces qui le composent. Seules les feuilles portent de la matière achetée ; les nœuds intermédiaires ne sont que des regroupements.

La quantité cumulée ne peut pas s'obtenir par une somme, parce que les quantités d'une nomenclature sont relatives à leur assemblage immédiat. Deux portes, chacune montée avec quatre vis, demandent huit vis, pas six. La composition le long d'un chemin est multiplicative, ce qui se code en traînant la quantité dans la CTE et en écrivant `n.qte_cumulee * b.quantite` dans la partie récursive. Une somme donnerait un nombre plausible, donc une erreur qui ne se voit pas.

Si un composant apparaît à deux endroits de l'arbre, il ne faut surtout pas le dédoublonner : les deux apparitions sont deux besoins réels, et c'est le total qui compte. On le laisse donc figurer deux fois dans la CTE, avec sa quantité cumulée propre à chaque chemin, et on ne regroupe qu'à la toute fin, par un `GROUP BY` sur la pièce avec une somme. L'ordre est essentiel : regrouper avant d'avoir fini de cumuler mélangerait des chemins de profondeurs différentes.

Théorie : `Theorie_05_SQL.md`, sections 7 et 9.

M2. Réseau de cliniques Rive-Sud.

Le cœur de la solution est qu'une date se glisse dans une CTE récursive comme une condition sur les colonnes de validité, appliquée dans les deux morceaux. La bonne façon de la passer une seule fois est de la poser dans une CTE de tête, `parametres AS (SELECT DATE '2025-01-01' AS d)`, jointe par un `CROSS JOIN`

© 2026 iAlexMG. Tous droits réservés.

dans l'ancre et dans la partie récursive. La répéter en dur aux deux endroits marche aussi, et c'est exactement le genre de duplication qui finit par diverger : quelqu'un changera un jour la date de l'ancre en oubliant l'autre, et la requête produira un organigramme mi-ancien mi-nouveau, cohérent en apparence.

L'organigramme à une date donnée est donc une récursion qui ne suit que les liens valides à cette date, c'est-à-dire `debut <= d AND (fin IS NULL OR fin > d)`. On l'exécute deux fois, une par date, en deux CTE distinctes, puis on compare les niveaux obtenus par une jointure sur l'identifiant de la personne.

Les personnes qui n'existent que d'un côté sont le vrai sujet de la troisième demande, et il ne faut pas les traiter comme du bruit. Une jointure interne entre les deux organigrammes les ferait disparaître silencieusement, et la direction lirait une liste de changements qui omet les mouvements les plus significatifs, les départs et les arrivées. Il faut donc un `FULL OUTER JOIN` entre les deux états, puis trois catégories explicites : présent des deux côtés avec un niveau différent, présent seulement avant, présent seulement après. Nommer les trois cas dans le résultat vaut mieux que de les laisser deviner à partir de colonnes nulles.

Sur la question qui décide de tout, cette requête doit vivre dans un entrepôt, et pour une raison qui n'est pas la performance. Neuf cents employés, ce n'est rien à parcourir, même en récursif : l'application encaisserait la charge sans broncher. Ce qui tranche, c'est que la question est historique. Elle demande un état passé, or une base applicative est faite pour représenter le présent et n'a aucune obligation de conserver proprement ses états antérieurs : une correction de saisie, une purge, une reprise de données, et l'organigramme de l'an dernier n'est plus reconstituable. Un entrepôt, lui, a précisément pour métier d'historiser. Je changerais d'avis si la table du personnel portait déjà un historique complet et garanti, auquel cas la requête pourrait vivre en lecture sur une réplique.

Théorie : `Theorie_05_SQL.md`, section 7. Théorie : `Theorie_02_SQL.md`, sections 2 et 6.

M3. `(avancé)` Coopérative de transport Chaudière.

La stratégie est une CTE récursive dont l'ancre est le point de départ et dont la partie récursive suit les arêtes, en accumulant le nombre d'étapes et le chemin parcouru. Le `SELECT` final ne garde que les lignes arrivées au point demandé.

Le cœur du problème est le cycle. Sur un graphe qui contient des boucles, la partie récursive trouve toujours une arête à suivre, donc elle ne se tarit jamais et la requête ne s'arrête pas d'elle-même. Deux mécanismes de protection existent, et ils ne sont pas équivalents.

Le premier borne la profondeur, `WHERE n.etapes < 10`. Il garantit l'arrêt, ce qui est déjà beaucoup, mais il ne garantit rien sur la justesse : il coupe au milieu d'une exploration, donc il renvoie une partie des chemins et en tait d'autres, sans dire lesquels. C'est une protection de survie, pas une solution.

Le second traîne le chemin déjà parcouru, comme un tableau d'identifiants ou une chaîne de clés, et refuse d'ajouter un point qui s'y trouve déjà. Sur PostgreSQL, cela s'écrit avec un tableau et l'opérateur `<> ALL`, ou avec `NOT (b.arrivee = ANY(n.chemin))`. C'est le seul mécanisme correct, parce qu'il exprime exactement la contrainte du problème, à savoir qu'un trajet ne repasse pas par un point déjà visité. Il termine toujours, puisque le nombre de points est fini, et il ne perd aucun chemin légitime. En pratique on garde quand même la borne de profondeur par-dessus, non pas comme protection mais comme garde-fou : si elle se déclenche, c'est que le graphe est plus dense qu'on ne le croyait, et l'information vaut d'être remontée.

Sur le coût, le renversement est net, et il est mesurable. Cette requête est petite en volume mais profondément itérative, avec de nombreux allers-retours sur une même petite table. C'est le profil qui convient à un moteur transactionnel, où un index sur la colonne de départ rend chaque tour quasi immédiat. Un moteur en colonnes est mal servi ici : il excelle à balayer beaucoup de lignes en une passe, pas à répéter des milliers de tours minuscules, et il paie un prix fixe à chacun. La même CTE récursive de 100 000 tours prend 42 ms sur PostgreSQL et 9,5 s sur DuckDB, deux cents fois plus, et l'écart croît avec le nombre de tours

© 2026 iAlexMG. Tous droits réservés.

puisque c’est le tour qui coûte. C’est l’un des rares cas de la formation où la voie transactionnelle est la bonne pour des raisons de performance et non seulement de fraîcheur.

Reste la question qui fâche. Le SQL récursif est le bon outil tant que le graphe est petit et que la question reste ponctuelle, disons quelques milliers d’arêtes et un besoin d’exploration occasionnel. Au-delà, l’explosion combinatoire du nombre de chemins, qui n’a rien à voir avec le nombre d’arêtes, rend l’énumération exhaustive absurde : demander tous les trajets entre deux points d’un réseau dense revient à demander un résultat plus gros que la base. Je changerais d’outil au moment où la question elle-même change de nature, c’est-à-dire quand on cesse de vouloir tous les trajets pour ne vouloir que le meilleur. Un plus court chemin se calcule par un algorithme dédié, dans une base orientée graphe ou dans du code applicatif, et le SQL n’a pas à faire semblant.

Théorie : `Theorie_05_SQL.md`, sections 5 et 8. Théorie : `Theorie_02_SQL.md`, section 2.

© 2026 iAlexMG. Tous droits réservés.